```
# data manupilation
import pandas as pd
import numpy as np
```

```
# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Preprocessing tools
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```
# Machine Learning Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

```
# Evaluation metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    roc_curve
)
```

```
# Class imbalance handling
from imblearn.over_sampling import SMOTE
# Model interpretation
import shap
```

```
# Load the dataset
# Read the CSV file
df = pd.read_csv('hmeq.csv')
print(f"   Shape: {df.shape[0]} rows (borrowers) × {df.shape[1]} columns (features)")
```

```
   Shape: 5960 rows (borrowers) × 13 columns (features)
```

```
print(df.head())
```

```
   BAD  LOAN  MORTDUE     VALUE   REASON     JOB   YOJ  DEROG  DELINQ  \
0    1  1100  25860.0   39025.0  HomeImp   Other  10.5    0.0     0.0
1    1  1300  70053.0   68400.0  HomeImp   Other   7.0    0.0     2.0
2    1  1500  13500.0   16700.0  HomeImp   Other   4.0    0.0     0.0
3    1  1500      NaN       NaN      NaN     NaN   NaN    NaN     NaN
4    0  1700  97800.0  112000.0  HomeImp  Office   3.0    0.0     0.0

        CLAGE  NINQ  CLNO  DEBTINC
```

```
0   94.366667   1.0   9.0      NaN
1  121.833333   0.0  14.0      NaN
2  149.466667   1.0  10.0      NaN
3         NaN   NaN   NaN      NaN
4   93.333333   0.0  14.0      NaN
```

```
#Basic Information
print(f"Total number of borrowers: {df.shape[0]}")
print(f"Total number of features: {df.shape[1]}")
print(f"\nColumn names: {df.columns.tolist()}")
```

```
Total number of borrowers: 5960
Total number of features: 13

Column names: ['BAD', 'LOAN', 'MORTDUE', 'VALUE', 'REASON', 'JOB', 'YOJ', 'DEROG', 'DELINQ', 'CLAGE', 'NINQ', 'CLNO', 'DEBTINC']
```

```
print(df.dtypes)
```

```
BAD          int64
LOAN         int64
MORTDUE    float64
VALUE      float64
REASON      object
JOB         object
YOJ        float64
DEROG      float64
DELINQ     float64
CLAGE      float64
NINQ       float64
CLNO       float64
DEBTINC    float64
dtype: object
```

```
# Missing Values
print("\n---- Missing Values Check ---")
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100

missing_df = pd.DataFrame({
    'Column': missing.index,
    'Missing_Count': missing.values,
    'Percentage': missing_percent.values
})
missing_df = missing_df[missing_df['Missing_Count'] > 0].sort_values('Missing_Count', ascending=False)

print(missing_df.to_string(index=False))
```

```
---- Missing Values Check ---
 Column  Missing_Count  Percentage
DEBTINC           1267   21.258389
  DEROG            708   11.879195
 DELINQ            580    9.731544
```

```
MORTDUE            518    8.691275
    YOJ            515    8.640940
   NINQ            510    8.557047
  CLAGE            308    5.167785
    JOB            279    4.681208
 REASON            252    4.228188
   CLNO            222    3.724832
  VALUE            112    1.879195
```

```
# Statistical Summary
print(df.describe())
```

```
                BAD          LOAN        MORTDUE          VALUE          YOJ  \
count  5960.000000   5960.000000    5442.000000    5848.000000  5445.000000
mean      0.199497  18607.969799   73760.817200  101776.048741     8.922268
std       0.399656  11207.480417   44457.609458   57385.775334     7.573982
min       0.000000   1100.000000    2063.000000    8000.000000     0.000000
25%       0.000000  11100.000000   46276.000000   66075.500000     3.000000
50%       0.000000  16300.000000   65019.000000   89235.500000     7.000000
75%       0.000000  23300.000000   91488.000000  119824.250000    13.000000
max       1.000000  89900.000000  399550.000000  855909.000000    41.000000

             DEROG       DELINQ        CLAGE         NINQ         CLNO  \
count  5252.000000  5380.000000  5652.000000  5450.000000  5738.000000
mean      0.254570     0.449442   179.766275     1.186055    21.296096
std       0.846047     1.127266    85.810092     1.728675    10.138933
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000   115.116702     0.000000    15.000000
50%       0.000000     0.000000   173.466667     1.000000    20.000000
75%       0.000000     0.000000   231.562278     2.000000    26.000000
max      10.000000    15.000000  1168.233561    17.000000    71.000000

           DEBTINC
count  4693.000000
mean     33.779915
std       8.601746
min       0.524499
25%      29.140031
50%      34.818262
75%      39.003141
max     203.312149
```

```
#Target Variable Distribution
target_counts = df['BAD'].value_counts()
target_percent = df['BAD'].value_counts(normalize=True) * 100
```

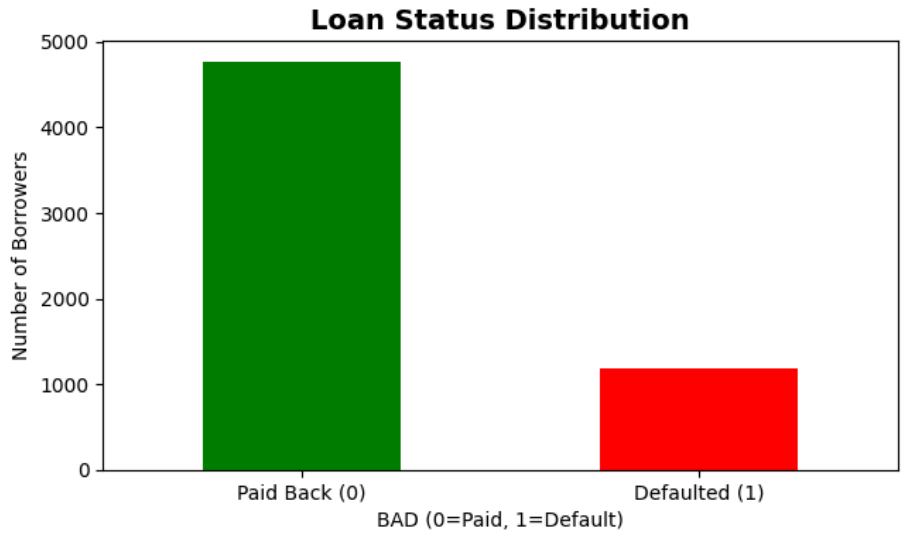
```
print("\nCount:")
print(target_counts)
print("\nPercentage:")
print(target_percent)
```

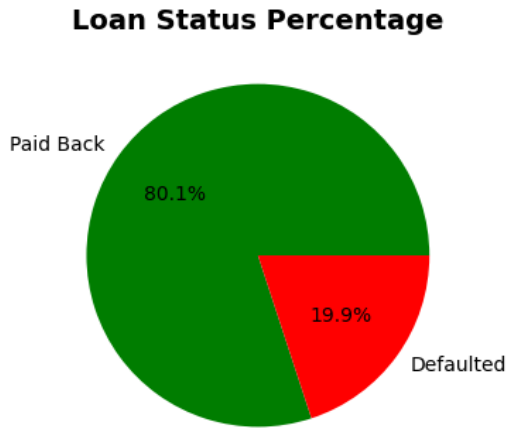
```

Count:
BAD
0    4771
```

```
1    1189
Name: count, dtype: int64

Percentage:
BAD
0    80.050336
1    19.949664
Name: proportion, dtype: float64
```

```
# Visualize target distribution
import matplotlib.pyplot as plt
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
 #Bar plot
target_counts.plot(kind='bar', ax=axes[0], color=['green', 'red'])
axes[0].set_title('Loan Status Distribution', fontsize=14, fontweight='bold')
axes[0].set_xlabel('BAD (0=Paid, 1=Default)')
axes[0].set_ylabel('Number of Borrowers')
axes[0].set_xticklabels(['Paid Back (0)', 'Defaulted (1)'], rotation=0)

target_counts.plot(
    kind='pie',           # Pie chart
    ax=axes[1],           # Put it in the right space
    autopct='%1.1f%%',    # Show percentages with 1 decimal
    colors=['green', 'red'],  # Colors for slices
    labels=['Paid Back', 'Defaulted']  # Labels for slices
)
axes[1].set_title('Loan Status Percentage', fontsize=14, fontweight='bold')
axes[1].set_ylabel('')  # Remove default ylabel
print("✓ Pie chart added to right side")

# STEP 4: Adjust layout and SHOW the figure
plt.tight_layout()  # Make sure charts don't overlap
plt.show()  # THIS IS CRITICAL - displays the window!
```

```
✓ Pie chart added to right side
```






```
#Separate features (X) from target (y)
X = df.drop('BAD', axis=1)  # All columns except 'BAD'
y = df['BAD']
print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
```

```
Features (X) shape: (5960, 12)
Target (y) shape: (5960,)
```

```
#Identify Column Types
print("\n---Identifying Column Types ---")

numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()

print(f"\nNumerical columns ({len(numerical_cols)}):")
print(numerical_cols)
print(f"\nCategorical columns ({len(categorical_cols)}):")
print(categorical_cols)
```

```
---Identifying Column Types ---

Numerical columns (10):
['LOAN', 'MORTDUE', 'VALUE', 'YOJ', 'DEROG', 'DELINQ', 'CLAGE', 'NINQ', 'CLNO', 'DEBTINC']

Categorical columns (2):
['REASON', 'JOB']
```

```
#Handle Missing Values
print("\n--- Handling Missing Values ---")

print("\nBefore imputation:")
print(X.isnull().sum()[X.isnull().sum() > 0])

# For numerical columns: Fill with mean (average)
if len(numerical_cols) > 0:
    num_imputer = SimpleImputer(strategy='mean')
    X[numerical_cols] = num_imputer.fit_transform(X[numerical_cols])
    print(f"\n✓ Filled missing values in numerical columns with MEAN")

# For categorical columns: Fill with mode (most common)
if len(categorical_cols) > 0:
    cat_imputer = SimpleImputer(strategy='most_frequent')
    X[categorical_cols] = cat_imputer.fit_transform(X[categorical_cols])
    print(f"✓ Filled missing values in categorical columns with MODE (most common)")
```

```
--- Handling Missing Values ---

Before imputation:
Series([], dtype: int64)

✓ Filled missing values in numerical columns with MEAN
✓ Filled missing values in categorical columns with MODE (most common)
```

```
# Encode Categorical Variables
print("\n--- Encoding Categorical Variables ---")

# Re-identify categorical columns based on the current state of X
categorical_cols_to_encode = X.select_dtypes(include=['object']).columns.tolist()

print("\nBefore encoding:")
print(f"Number of columns: {X.shape[1]}")
print(f"Categorical columns identified for encoding: {categorical_cols_to_encode}")

if categorical_cols_to_encode:
    # One-Hot Encoding: Convert categories to binary columns
    X_encoded = pd.get_dummies(X, columns=categorical_cols_to_encode, drop_first=True)

    print("\nAfter encoding:")
    print(f"Number of columns: {X_encoded.shape[1]}")
    # Display new columns created more robustly
    newly_created_cols = [col for col in X_encoded.columns if col not in X.columns]
    print(f"New columns created: {newly_created_cols}")

    # Update X to encoded version
    X = X_encoded
else:
    print("\nNo categorical columns found for encoding. Skipping encoding step.")
    # If no encoding is needed, X is already in its final state for this step
    X_encoded = X
```

```
print("   One-Hot Encoding converts categories to binary (0/1) columns\n")

print(X.head())
```

```
--- Encoding Categorical Variables ---

Before encoding:
Number of columns: 16
Categorical columns identified for encoding: []

No categorical columns found for encoding. Skipping encoding step.
   One-Hot Encoding converts categories to binary (0/1) columns

     LOAN     MORTDUE          VALUE        YOJ    DEROG    DELINQ  \
0  1100.0  25860.0000   39025.000000  10.500000  0.00000  0.000000
1  1300.0  70053.0000   68400.000000   7.000000  0.00000  2.000000
2  1500.0  13500.0000   16700.000000   4.000000  0.00000  0.000000
3  1500.0  73760.8172  101776.048741   8.922268  0.25457  0.449442
4  1700.0  97800.0000  112000.000000   3.000000  0.00000  0.000000

        CLAGE      NINQ       CLNO    DEBTINC  REASON_HomeImp  JOB_Office  \
0   94.366667  1.000000   9.000000  33.779915            True       False
1  121.833333  0.000000  14.000000  33.779915            True       False
2  149.466667  1.000000  10.000000  33.779915            True       False
3  179.766275  1.186055  21.296096  33.779915           False       False
4   93.333333  0.000000  14.000000  33.779915            True        True

   JOB_Other  JOB_ProfExe  JOB_Sales  JOB_Self
0       True        False      False     False
1       True        False      False     False
2       True        False      False     False
3       True        False      False     False
4      False        False      False     False
```

```
# Split the data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,        # 20% for testing
    random_state=42,      # For reproducibility (same split every time)
    stratify=y            # Keep same proportion of defaults in both sets
)

print(f"\nOriginal dataset size: {len(X)} borrowers")
print(f"\nTraining set: {len(X_train)} borrowers ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing set:  {len(X_test)} borrowers ({len(X_test)/len(X)*100:.0f}%)")

print("\n--- Class Distribution in Training Set ---")
train_dist = pd.Series(y_train).value_counts()
print(f"Paid back (0): {train_dist[0]} ({train_dist[0]/len(y_train)*100:.1f}%)")
print(f"Defaulted (1): {train_dist[1]} ({train_dist[1]/len(y_train)*100:.1f}%)")
```

```
print("\n--- Class Distribution in Testing Set ---")
test_dist = pd.Series(y_test).value_counts()
print(f"Paid back (0): {test_dist[0]} ({test_dist[0]/len(y_test)*100:.1f}%)")
print(f"Defaulted (1): {test_dist[1]} ({test_dist[1]/len(y_test)*100:.1f}%)")
```

```
Original dataset size: 5960 borrowers

Training set: 4768 borrowers (80%)
Testing set:  1192 borrowers (20%)

--- Class Distribution in Training Set ---
Paid back (0): 3817 (80.1%)
Defaulted (1): 951 (19.9%)

--- Class Distribution in Testing Set ---
Paid back (0): 954 (80.0%)
Defaulted (1): 238 (20.0%)
```

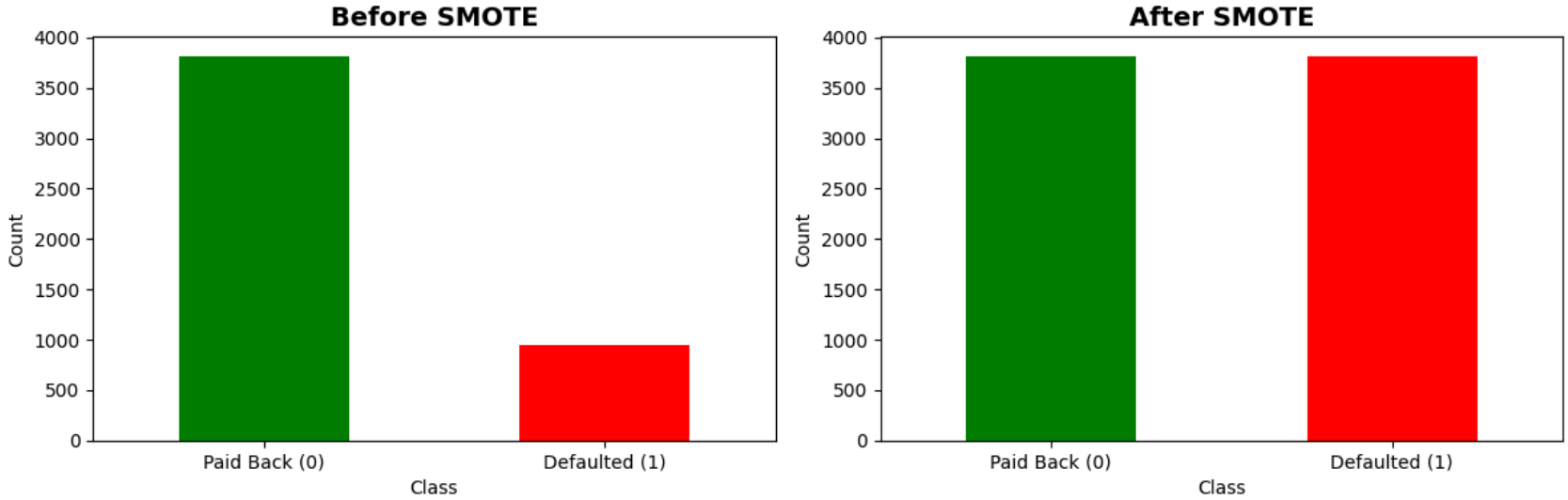
```
print("\n--- Before Scaling ---")
print("Sample of training data (first 3 rows, first 5 columns):")
print(pd.DataFrame(X_train, columns=X.columns).iloc[:3, :5])

# Initialize the scaler
scaler = StandardScaler()

# Fit on training data and transform both sets
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\n--- After Scaling ---")
print("Sample of scaled training data (first 3 rows, first 5 columns):")
print(pd.DataFrame(X_train_scaled, columns=X.columns).iloc[:3, :5])
```

```
--- Before Scaling ---
Sample of training data (first 3 rows, first 5 columns):
         LOAN  MORTDUE    VALUE  YOJ    DEROG
609    7700.0  70451.0  81862.0  3.0  0.00000
4015  21000.0  48735.0  71694.0  8.0  0.00000
1591  11500.0  63136.0  81099.0  3.0  0.25457

--- After Scaling ---
Sample of scaled training data (first 3 rows, first 5 columns):
       LOAN   MORTDUE     VALUE       YOJ     DEROG
0 -0.964574 -0.076575 -0.350518 -0.813080 -0.322664
1  0.219783 -0.590597 -0.530698 -0.125662 -0.322664
2 -0.626186 -0.249722 -0.364038 -0.813080 -0.002348
```

```
print("\n--- Before SMOTE ---")
```

```
print("Training set class distribution:")
before_counts = pd.Series(y_train).value_counts()
print(f"Paid back (0): {before_counts[0]}")
print(f"Defaulted (1): {before_counts[1]}")
print(f"Ratio: {before_counts[0]/before_counts[1]:.1f}:1")
print(f"Total samples: {len(y_train)}")

# Visualize before SMOTE
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

before_counts.plot(kind='bar', ax=axes[0], color=['green', 'red'])
axes[0].set_title('Before SMOTE', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Class')
axes[0].set_ylabel('Count')
axes[0].set_xticklabels(['Paid Back (0)', 'Defaulted (1)'], rotation=0)

# Apply SMOTE
print("\n--- Applying SMOTE ---")
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

print("✓ SMOTE applied successfully!")

print("\n--- After SMOTE ---")
print("Training set class distribution:")
after_counts = pd.Series(y_train_resampled).value_counts()
print(f"Paid back (0): {after_counts[0]}")
print(f"Defaulted (1): {after_counts[1]}")
print(f"Ratio: {after_counts[0]/after_counts[1]:.1f}:1")
print(f"Total samples: {len(y_train_resampled)}")

# Visualize after SMOTE
after_counts.plot(kind='bar', ax=axes[1], color=['green', 'red'])
axes[1].set_title('After SMOTE', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Class')
axes[1].set_ylabel('Count')
axes[1].set_xticklabels(['Paid Back (0)', 'Defaulted (1)'], rotation=0)

plt.tight_layout()
plt.show()
```

```
--- Before SMOTE ---
Training set class distribution:
Paid back (0): 3817
Defaulted (1): 951
Ratio: 4.0:1
Total samples: 4768

--- Applying SMOTE ---
✓ SMOTE applied successfully!

--- After SMOTE ---
Training set class distribution:
Paid back (0): 3817
Defaulted (1): 3817
Ratio: 1.0:1
Total samples: 7634
```




```
# model training
#logistic regression
print("\nTraining Logistic Regression...")
lr_model = LogisticRegression(random_state=42, max_iter=1000)
lr_model.fit(X_train_resampled, y_train_resampled)
print("✓ Logistic Regression trained successfully!")

# Cross-validation
print("\nPerforming 5-Fold Cross-Validation...")
lr_cv_scores = cross_val_score(lr_model, X_train_resampled, y_train_resampled,
                               cv=5, scoring='roc_auc')
print(f"CV AUC-ROC Scores: {lr_cv_scores}")
print(f"Mean CV AUC-ROC: {lr_cv_scores.mean():.4f} (+/- {lr_cv_scores.std():.4f})")
```

```
Training Logistic Regression...
✓ Logistic Regression trained successfully!

Performing 5-Fold Cross-Validation...
CV AUC-ROC Scores: [0.81289927 0.82132736 0.79238745 0.81771287 0.81102566]
Mean CV AUC-ROC: 0.8111 (+/- 0.0100)
```

```
print("\nTraining Random Forest...")
rf_model = RandomForestClassifier(
    n_estimators=100,     # Number of trees
    max_depth=20,         # Maximum depth of each tree
    random_state=42,
    n_jobs=-1            # Use all CPU cores
)
rf_model.fit(X_train_resampled, y_train_resampled)
print("✓ Random Forest trained successfully!")

# Cross-validation
print("\nPerforming 5-Fold Cross-Validation...")
rf_cv_scores = cross_val_score(rf_model, X_train_resampled, y_train_resampled,
                               cv=5, scoring='roc_auc')
print(f"CV AUC-ROC Scores: {rf_cv_scores}")
print(f"Mean CV AUC-ROC: {rf_cv_scores.mean():.4f} (+/- {rf_cv_scores.std():.4f})")
```

```
Training Random Forest...
✓ Random Forest trained successfully!

Performing 5-Fold Cross-Validation...
CV AUC-ROC Scores: [0.98582425 0.99331826 0.99362533 0.99381575 0.99536561]
Mean CV AUC-ROC: 0.9924 (+/- 0.0034)
```

```
# XG BOOST
print("\nTraining XGBoost...")
xgb_model = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    random_state=42,
    eval_metric='logloss'
)
xgb_model.fit(X_train_resampled, y_train_resampled)
print("✓ XGBoost trained successfully!")

# Cross-validation
print("\nPerforming 5-Fold Cross-Validation...")
xgb_cv_scores = cross_val_score(xgb_model, X_train_resampled, y_train_resampled,
                                cv=5, scoring='roc_auc')
print(f"CV AUC-ROC Scores: {xgb_cv_scores}")
print(f"Mean CV AUC-ROC: {xgb_cv_scores.mean():.4f} (+/- {xgb_cv_scores.std():.4f})")
```

```
Training XGBoost...
✓ XGBoost trained successfully!

Performing 5-Fold Cross-Validation...
CV AUC-ROC Scores: [0.95485408 0.98144724 0.9836945  0.98548716 0.98662072]
Mean CV AUC-ROC: 0.9784 (+/- 0.0119)
```

```
print("\n--- Cross-Validation Summary ---")
print(f"Logistic Regression CV AUC: {lr_cv_scores.mean():.4f}")
print(f"Random Forest CV AUC:       {rf_cv_scores.mean():.4f}")
print(f"XGBoost CV AUC:             {xgb_cv_scores.mean():.4f}")
```

```
--- Cross-Validation Summary ---
Logistic Regression CV AUC: 0.8111
Random Forest CV AUC:       0.9924
XGBoost CV AUC:             0.9784
```

```
# model evaluation
def evaluate_model(model, X_test, y_test, model_name):
    """Evaluate a model and return results"""


    # Make predictions
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[:, 1]

    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_pred_proba)

    print(f"\n PERFORMANCE METRICS:")
    print(f"   Accuracy:  {accuracy:.4f}")
    print(f"   Precision: {precision:.4f}")
    print(f"   Recall:    {recall:.4f}")
    print(f"   F1-Score:  {f1:.4f}")
    print(f"   AUC-ROC:   {roc_auc:.4f}")

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    print(f"\n CONFUSION MATRIX:")
    print(f"                  Predicted")
    print(f"                  0      1")
    print(f"   Actual   0  [{cm[0,0]:4d}  {cm[0,1]:4d}]")
    print(f"            1  [{cm[1,0]:4d}  {cm[1,1]:4d}]")

    print(f"\nCONFUSION MATRIX EXPLANATION:")
    print(f"   True Negatives (TN):  {cm[0,0]} - Correctly predicted as paid back")
```

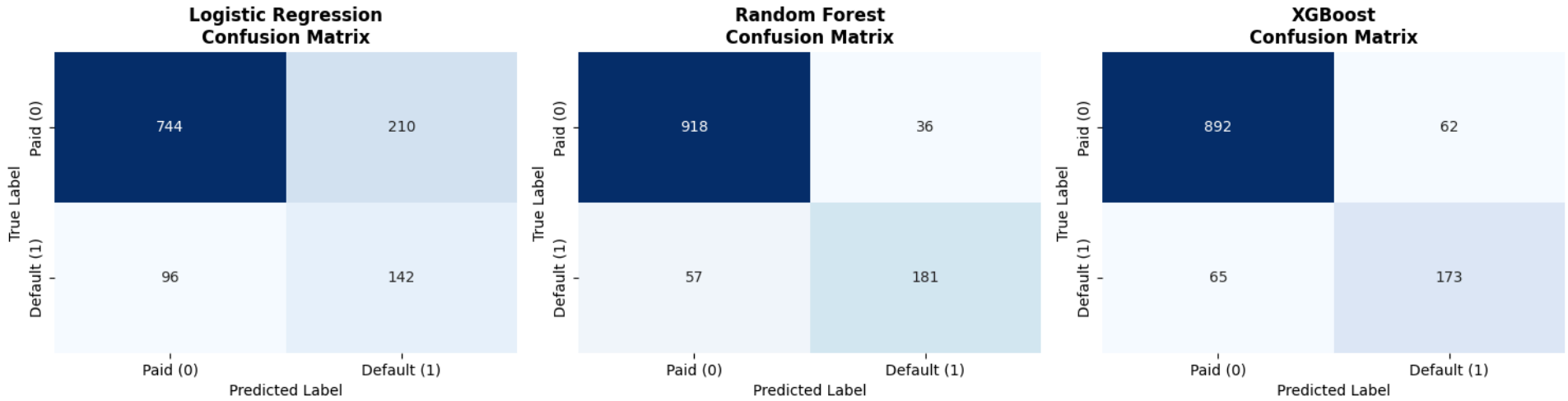
```
    print(f"   False Positives (FP): {cm[0,1]} - Wrongly predicted as default")
    print(f"   False Negatives (FN): {cm[1,0]} - Wrongly predicted as paid back (RISKY!)")
    print(f"   True Positives (TP):  {cm[1,1]} - Correctly predicted as default")


    # Classification Report
    print(f"\n DETAILED CLASSIFICATION REPORT:")
    print(classification_report(y_test, y_pred, target_names=['Paid Back (0)', 'Default (1)']))

    return {
        'model_name': model_name,
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'roc_auc': roc_auc,
        'confusion_matrix': cm,
        'y_pred_proba': y_pred_proba
    }

# Evaluate all three models
lr_results = evaluate_model(lr_model, X_test_scaled, y_test, "Logistic Regression")
rf_results = evaluate_model(rf_model, X_test_scaled, y_test, "Random Forest")
xgb_results = evaluate_model(xgb_model, X_test_scaled, y_test, "XGBoost")
```

```
 PERFORMANCE METRICS:
   Accuracy:  0.7433
   Precision: 0.4034
   Recall:    0.5966
   F1-Score:  0.4814
   AUC-ROC:   0.7531

 CONFUSION MATRIX:
                  Predicted
                  0      1
   Actual   0  [ 744   210]
            1  [  96   142]

CONFUSION MATRIX EXPLANATION:
   True Negatives (TN):  744 - Correctly predicted as paid back
   False Positives (FP): 210 - Wrongly predicted as default
   False Negatives (FN): 96 - Wrongly predicted as paid back (RISKY!)
   True Positives (TP):  142 - Correctly predicted as default

 DETAILED CLASSIFICATION REPORT:
               precision    recall  f1-score   support

Paid Back (0)       0.89      0.78      0.83       954
  Default (1)       0.40      0.60      0.48       238

     accuracy                           0.74      1192
    macro avg       0.64      0.69      0.66      1192
 weighted avg       0.79      0.74      0.76      1192
```

```
 PERFORMANCE METRICS:
   Accuracy:  0.9220
   Precision: 0.8341
   Recall:    0.7605
   F1-Score:  0.7956
   AUC-ROC:   0.9652

 CONFUSION MATRIX:
                  Predicted
                  0      1
   Actual   0  [ 918    36]
            1  [  57   181]

CONFUSION MATRIX EXPLANATION:
   True Negatives (TN):  918 - Correctly predicted as paid back
   False Positives (FP): 36 - Wrongly predicted as default
   False Negatives (FN): 57 - Wrongly predicted as paid back (RISKY!)
   True Positives (TP):  181 - Correctly predicted as default

 DETAILED CLASSIFICATION REPORT:
               precision    recall  f1-score   support

Paid Back (0)       0.94      0.96      0.95       954
  Default (1)       0.83      0.76      0.80       238

     accuracy                           0.92      1192
    macro avg       0.89      0.86      0.87      1192
```

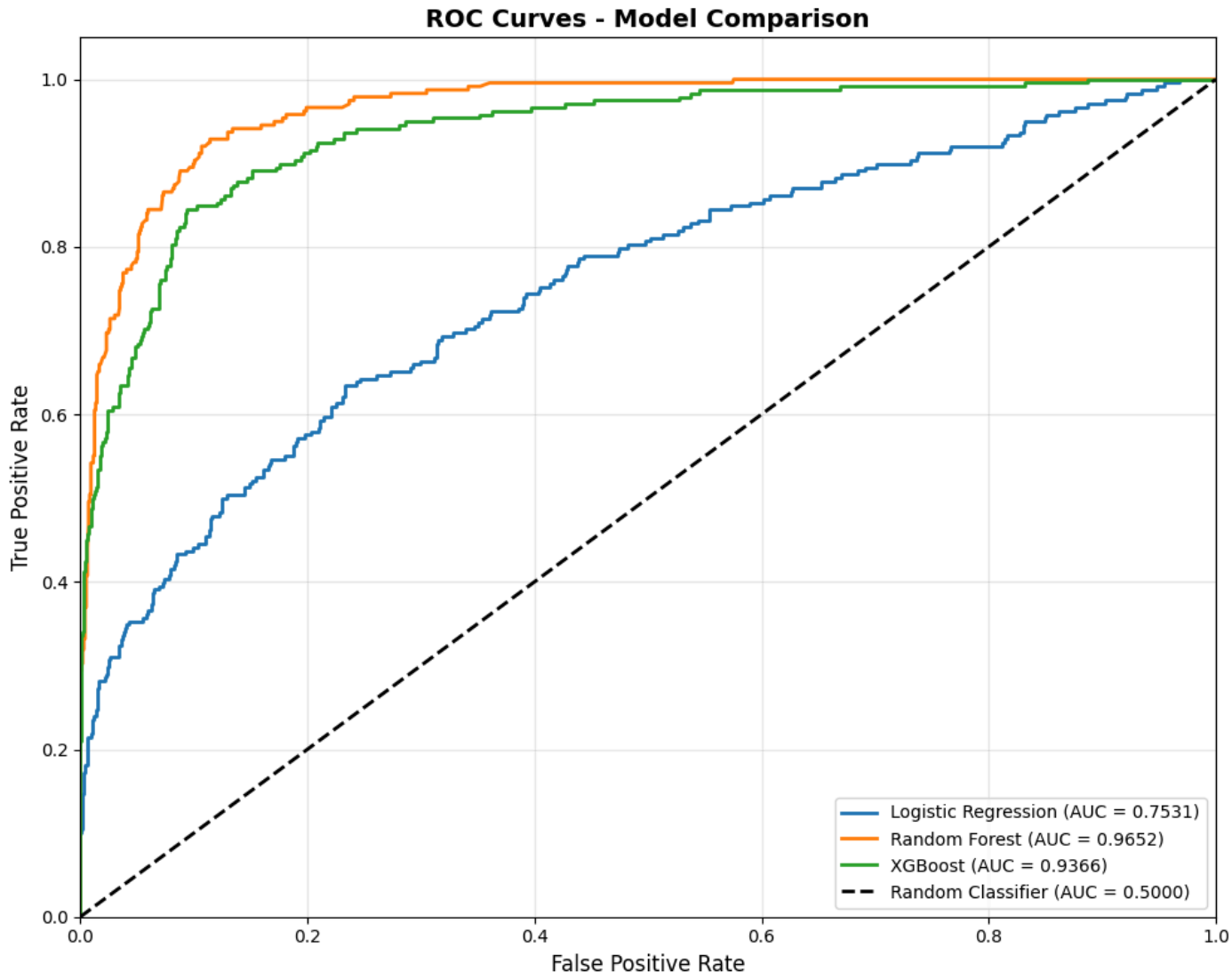
```
#Confusion Matrices
print("\n--- 10.1 Confusion Matrices ---")

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for idx, results in enumerate([lr_results, rf_results, xgb_results]):
    cm = results['confusion_matrix']
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[idx])
    axes[idx].set_title(f"{results['model_name']}\nConfusion Matrix",
                        fontsize=12, fontweight='bold')
    axes[idx].set_ylabel('True Label')
    axes[idx].set_xlabel('Predicted Label')
    axes[idx].set_xticklabels(['Paid (0)', 'Default (1)'])
    axes[idx].set_yticklabels(['Paid (0)', 'Default (1)'])

plt.tight_layout()
plt.show()

 #Darker colors = higher counts
  # We want high values on the diagonal (correct predictions
   #Low values off-diagonal (incorrect predictions
```

```
--- 10.1 Confusion Matrices ---
```



```
# 10.2 ROC Curves
print("\n--- 10.2 ROC Curves ---")

plt.figure(figsize=(10, 8))

# Plot ROC curve for each model
for results in [lr_results, rf_results, xgb_results]:
    fpr, tpr, _ = roc_curve(y_test, results['y_pred_proba'])
    plt.plot(fpr, tpr, lw=2, label=f"{results['model_name']} (AUC = {results['roc_auc']:.4f})")

# Plot random classifier line
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Random Classifier (AUC = 0.5000)')

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves - Model Comparison', fontsize=14, fontweight='bold')
plt.legend(loc="lower right", fontsize=10)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

#  X-axis = False Positive Rate (how many non-defaults wrongly predicted as default)
# Y-axis = True Positive Rate (how many defaults correctly identified)
#Closer to top-left corner = better model
#AUC (Area Under Curve) summarizes performance in one number
#AUC = 0.5 means random guessing
#AUC = 1.0 means perfect prediction
```

```
--- 10.2 ROC Curves ---
```

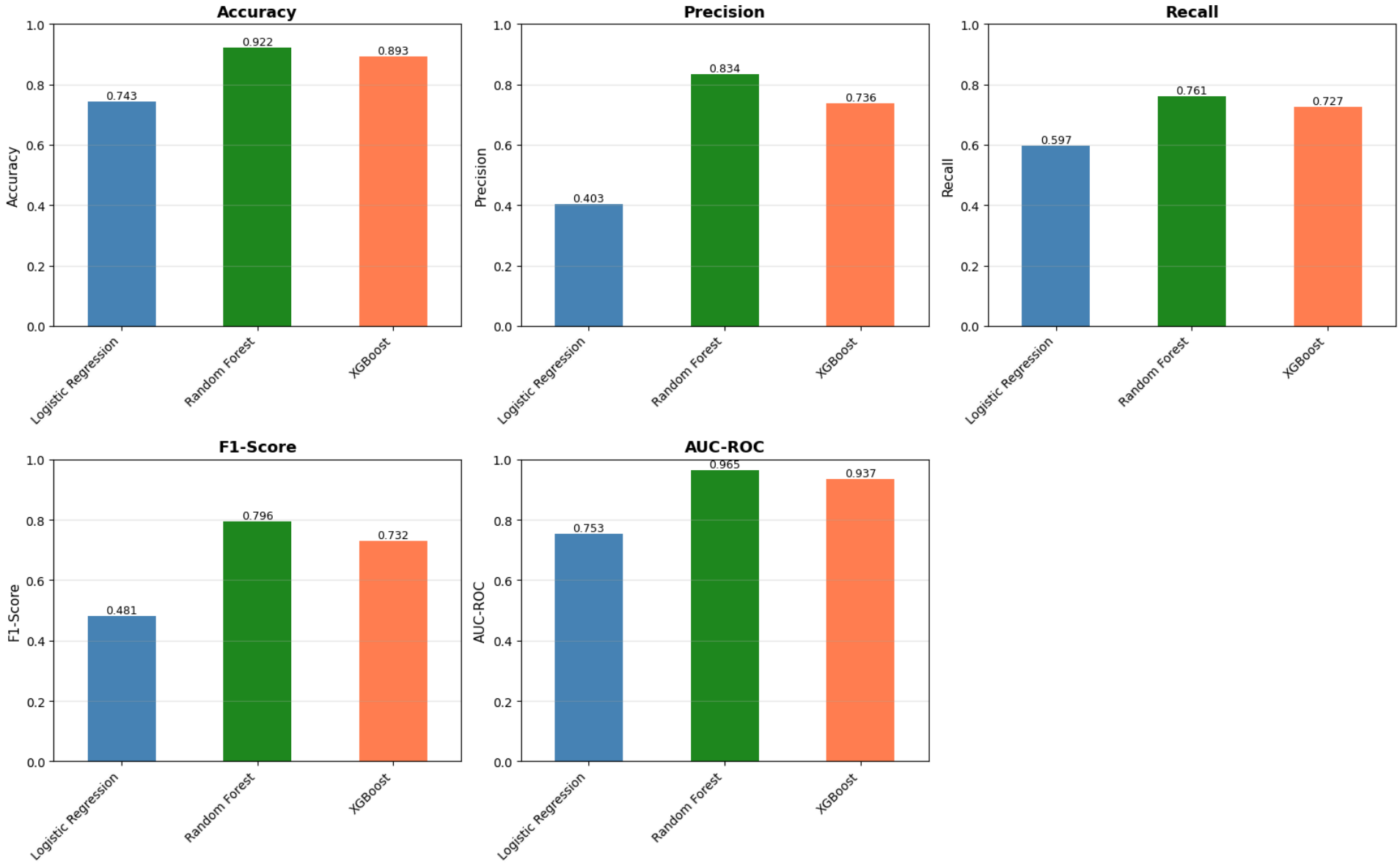



```
#  Metrics Comparison
print("\n--- 10.3 Metrics Comparison ---")

# Create comparison dataframe
comparison_df = pd.DataFrame([
    {
        'Model': r['model_name'],
        'Accuracy': r['accuracy'],
```

```
        'Precision': r['precision'],
        'Recall': r['recall'],
        'F1-Score': r['f1_score'],
        'AUC-ROC': r['roc_auc']
    }
    for r in [lr_results, rf_results, xgb_results]
])
print("\n PERFORMANCE COMPARISON TABLE:")
print(comparison_df.to_string(index=False))
```

```
--- 10.3 Metrics Comparison ---

📊 PERFORMANCE COMPARISON TABLE:
              Model  Accuracy  Precision   Recall  F1-Score  AUC-ROC
Logistic Regression  0.743289   0.403409 0.596639  0.481356 0.753070
      Random Forest  0.921980   0.834101 0.760504  0.795604 0.965215
            XGBoost  0.893456   0.736170 0.726891  0.731501 0.936552
```

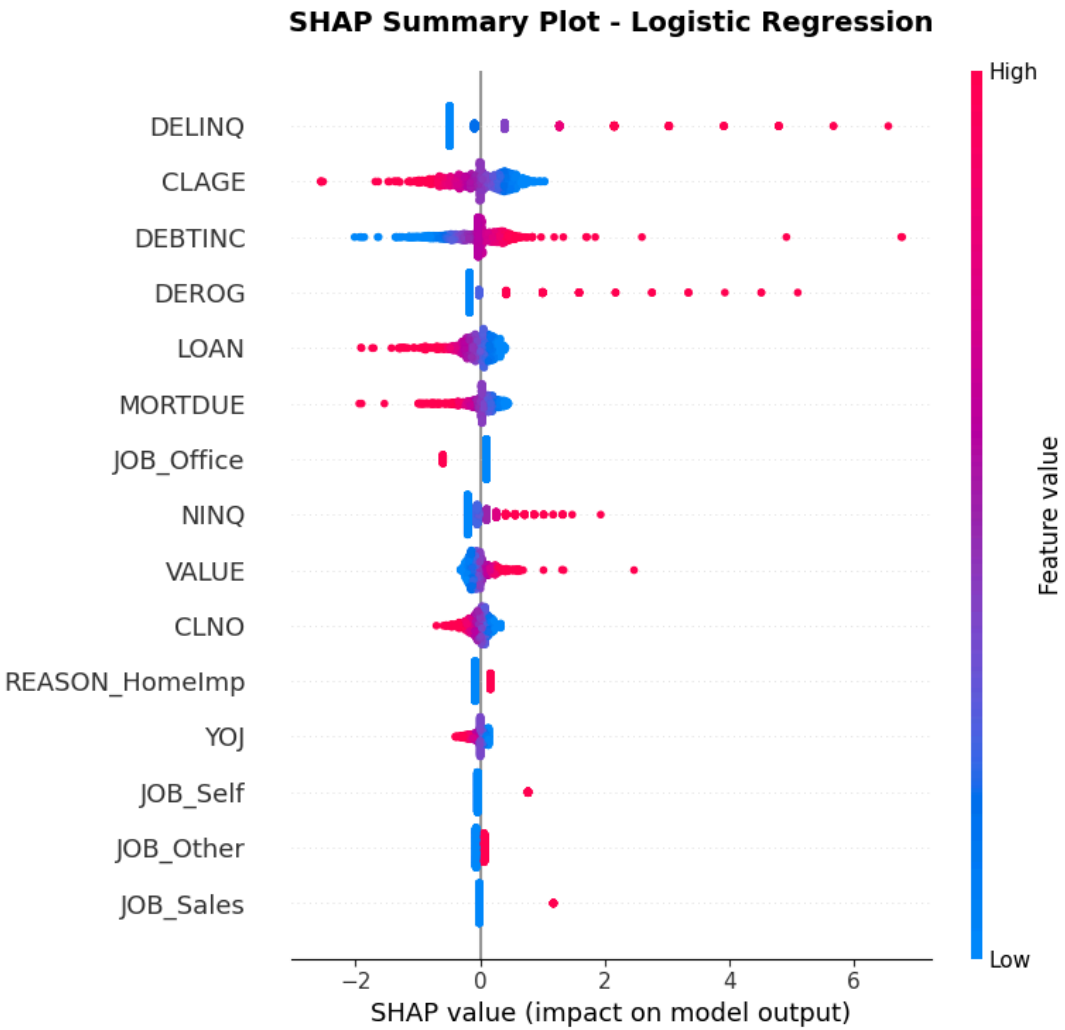
```
# Visualize comparison
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC-ROC']

for idx, metric in enumerate(metrics):
    row = idx // 3
    col = idx % 3
    ax = axes[row, col]

    comparison_df.plot(x='Model', y=metric, kind='bar', ax=ax, legend=False,
                       color=['steelblue', 'forestgreen', 'coral'])
    ax.set_title(metric, fontsize=13, fontweight='bold')
    ax.set_ylabel(metric, fontsize=11)
    ax.set_xlabel('')
    ax.set_ylim([0, 1])
    ax.grid(axis='y', alpha=0.3)
    ax.set_xticklabels(comparison_df['Model'], rotation=45, ha='right')

    # Add value labels on bars
    for container in ax.containers:
        ax.bar_label(container, fmt='%.3f', fontsize=9)

# Remove extra subplot
fig.delaxes(axes[1, 2])

plt.tight_layout()
plt.show()
```




```
# Get feature names
feature_names = X.columns.tolist()

# ===== 11.1 SHAP FOR LOGISTIC REGRESSION =====
```

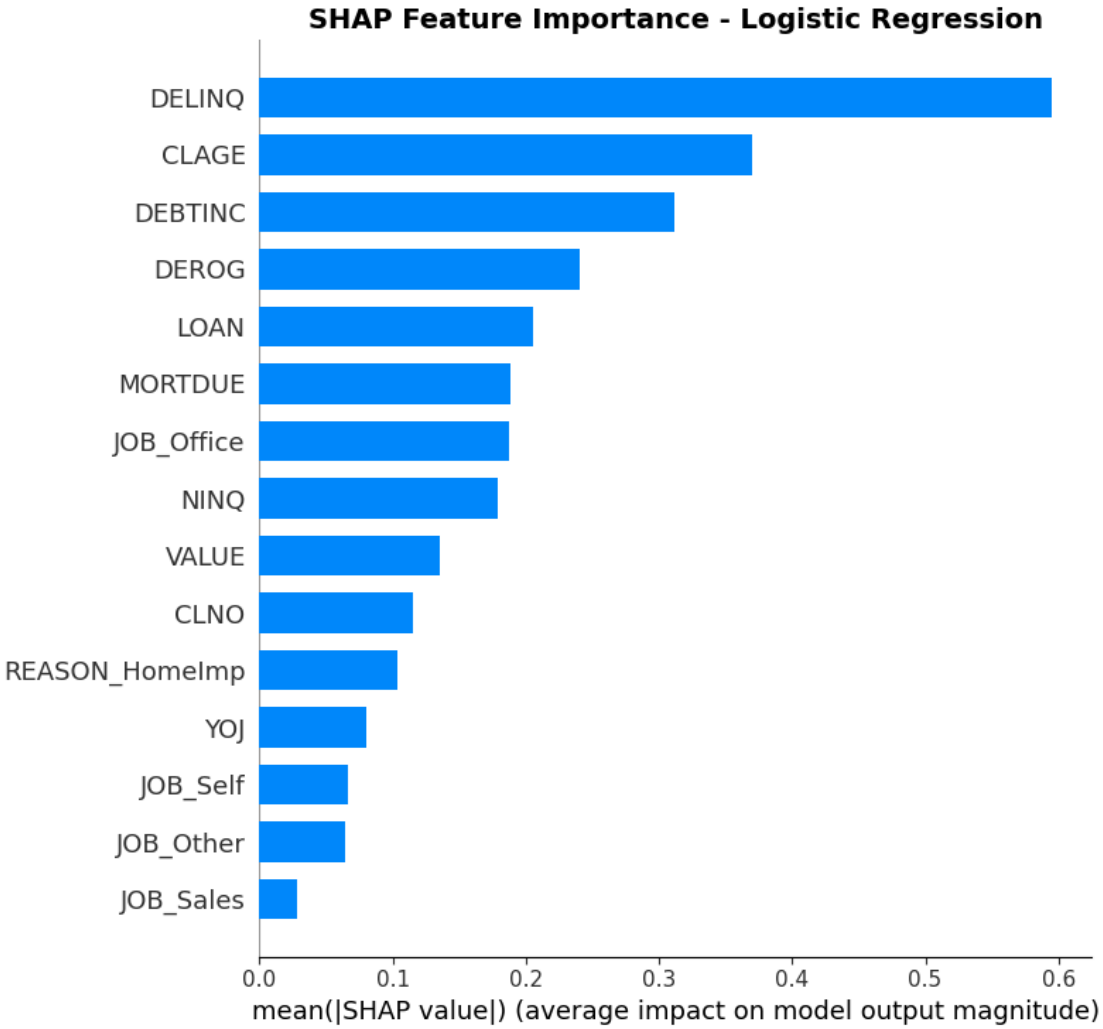
```
print("\nCalculating SHAP values for Logistic Regression...")
print("(This may take a minute...)")

lr_explainer = shap.LinearExplainer(lr_model, X_train_resampled)
lr_shap_values = lr_explainer.shap_values(X_test_scaled)

print("✓ SHAP values calculated!")

print("\n--- Summary Plot ---")
plt.figure(figsize=(10, 8))
shap.summary_plot(lr_shap_values, X_test_scaled, feature_names=feature_names,
                  max_display=15, show=False)
plt.title('SHAP Summary Plot - Logistic Regression', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
```

```
Calculating SHAP values for Logistic Regression...
(This may take a minute...)
✓ SHAP values calculated!

--- Summary Plot ---
```



```
print("\n--- Feature Importance Bar Plot ---")
plt.figure(figsize=(10, 8))
shap.summary_plot(lr_shap_values, X_test_scaled, feature_names=feature_names,
                  plot_type="bar", max_display=15, show=False)
plt.title('SHAP Feature Importance - Logistic Regression', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
--- Feature Importance Bar Plot ---
```




```
# ===== SHAP FOR RANDOM FOREST =====
```

```
print("\nCalculating SHAP values for Random Forest...")
print("(This may take a few minutes...)")

rf_explainer = shap.TreeExplainer(rf_model)
rf_shap_values = rf_explainer.shap_values(X_test_scaled)

# For binary classification, we use class 1 (default)
if isinstance(rf_shap_values, list):
    rf_shap_values = rf_shap_values[1]

print("✓ SHAP values calculated!")

print("\n--- Summary Plot ---")
plt.figure(figsize=(10, 8))
shap.summary_plot(rf_shap_values, X_test_scaled, feature_names=feature_names,
                  max_display=15, show=False)
plt.title('SHAP Summary Plot - Random Forest', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

print("\n--- Feature Importance Bar Plot ---")
plt.figure(figsize=(10, 8))
shap.summary_plot(rf_shap_values, X_test_scaled, feature_names=feature_names,
                  plot_type="bar", max_display=15, show=False)
plt.title('SHAP Feature Importance - Random Forest', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```